

UNIVERSIDAD DE BURGOS

REGIONAL UNIVERSITY NETWORK
EUROPEAN UNIVERSITY

GRADO EN INGENIERÍA INFORMÁTICA

Universidad de Burgos

Escuela Politécnica Superior

ASIGNATURA

Hardware de Aplicación Específica

TRABAJO / PRÁCTICA

Prácticas de laboratorio

Bloque 1

Estudiantes	Nombre y apellidos 1 Nombre y apellidos 2
Correo electrónico	correo@alu.ubu.es
Grupo	Grupo
Profesor/a	César Represa Pérez
Curso académico	2026–2027
Fecha de entrega	30 de octubre de 2026

Índice

1. Introducción **4**

2. Objetivos **4**

2.1. Objetivo general 4

2.2. Objetivos específicos 4

3. Entorno de trabajo **5**

3.1. Hardware y software 5

3.2. Imágenes utilizadas 5

3.3. Organización del código 6

4. Práctica 0: Introducción a MATLAB **6**

4.1. Planteamiento 6

4.2. Concatenación en las tres dimensiones 6

4.3. Operaciones con matrices 7

4.4. Búsqueda de valores y matrices aleatorias 7

5. Práctica 1: Procesado básico de imágenes en escala de grises **8**

5.1. Planteamiento 8

5.2. Validación de la entrada 8

5.3. Brillo 9

5.4. Ecualización del histograma 9

5.5. Umbralización 9

5.6. Suavizado 10

5.7. Gradiente 10

5.8. Realce 11

5.9. Medida de las botellas de `im4.jpg` 11

6. Práctica 2: Procesado de imágenes en color **13**

6.1. Planteamiento 13

6.2. Canales RGB 13
6.3. Espacios HSV e YCbCr 13
6.4. Selección de un color con `impixel` 14
6.5. Mapa de color 15
6.6. Color a partir de la rueda HSV 16
6.7. Diámetro de la moneda de `im10.jpg` 17

7. Práctica 3: Chroma Key **18**
7.1. Planteamiento 18
7.2. Mascarilla por distancia de color 19
7.3. Mascarilla por diferencia de color 19
7.4. Ajuste de la mascarilla 19

8. Práctica 4: Imágenes binarias **20**
8.1. Planteamiento 20
8.2. Segmentación 20
8.3. Visualización de la matriz de etiquetas 20
8.4. Rectángulos de área mínima y enumeración 20
8.5. Ranuras horizontales 21

9. Práctica 5: Reconocimiento y clasificación **21**
9.1. Planteamiento 21
9.2. Segmentación y etiquetado 22
9.3. Factor de forma 22
9.4. Textura 22
9.5. Espacio de características 22

10. Reto 1: ¿Cuántos euros hay? (opcional) **23**

11. Conclusiones **24**

12. Contribución de los integrantes **24**

13.Declaración sobre el uso de herramientas de inteligencia artificial **25**

14.Referencias **25**

A. Código fuente y repositorio **26**

B. Material adicional **26**

1. Introducción

El procesamiento digital de imagen consiste en la manipulación de una imagen almacenada como una matriz de números, de forma que se pueda mejorar su aspecto, extraer información medible de ella o reconocer automáticamente los objetos que contiene. Esta es una de las aplicaciones típicas del hardware de propósito específico: un sistema de visión artificial que inspecciona un producto en una cadena de montaje ejecuta exactamente este tipo de operaciones, y en la mayoría de los casos lo hace sobre un dispositivo dedicado en lugar de sobre un ordenador de propósito general.

La asignatura *Hardware de Aplicación Específica* aborda ese problema desde dos vertientes. En este primer bloque se trabaja la parte de tratamiento de la información: se utiliza MATLAB con la *Image Processing Toolbox* como entorno de prototipado para poner en práctica los algoritmos básicos de visión. En el segundo bloque se traslada ese mismo tipo de procesamiento a la adquisición y al vídeo en tiempo real.

El presente documento recoge el trabajo realizado en las prácticas 0 a 5 del bloque, más el reto 1 planteado al final del mismo: la introducción al entorno MATLAB, el procesado básico de imágenes en escala de grises, el procesado de imágenes en color, la técnica de *chroma key*, la segmentación de imágenes binarias y el reconocimiento y clasificación de objetos. Todo el código se ha desarrollado en MATLAB y se ha verificado con las imágenes proporcionadas en el enunciado. El material de referencia utilizado es el manual de la asignatura [1].

2. Objetivos

2.1. Objetivo general

Manejar las funciones básicas de procesamiento digital de imagen en MATLAB y aplicarlas a la resolución de problemas de inspección automática, comprendiendo en cada caso qué operación matemática se está realizando sobre la matriz de píxeles y qué información se puede extraer de ella.

2.2. Objetivos específicos

1. Familiarizarse con el manejo de vectores, matrices, *cell arrays* y estructuras de control en MATLAB.
2. Conocer la representación de una imagen digital y la influencia del tipo de dato (`uint8` frente a `double`) en los rangos de trabajo.

3. Aplicar operaciones de punto y de vecindad: brillo, ecualización del histograma, umbralización, filtrado espacial y detección de bordes.
4. Trabajar con los espacios de color RGB, HSV y YCbCr, identificando en qué casos conviene cada uno.
5. Segmentar objetos mediante umbralización, etiquetado de regiones y operaciones morfológicas.
6. Extraer descriptores geométricos y de textura para clasificar objetos y descartar falsos positivos.
7. Medir magnitudes reales a partir de la imagen, estableciendo la correspondencia entre píxeles y unidades físicas.

3. Entorno de trabajo

3.1. Hardware y software

Todo el bloque se ha desarrollado sobre MATLAB con la *Image Processing Toolbox*, que es la única caja de herramientas adicional que se necesita: las funciones `imread`, `imshow`, `imhist`, `imfilter`, `edge`, `bwlabel`, `regionprops`, `rgb2hsv`, `rgb2ycbcr` y `impixel`, entre otras, pertenecen a ella.

Cuadro 1: Entorno utilizado en las prácticas.

Elemento	**Descripción**
Sistema operativo	GNU/Linux
MATLAB	R2026b
Toolbox	Image Processing Toolbox
Editor	Editor de MATLAB
Documentación	Manual de la asignatura [1]

3.2. Imágenes utilizadas

El enunciado proporciona un conjunto de imágenes de prueba. Cada práctica trabaja con un subconjunto concreto, que se indica en su apartado correspondiente:

Cuadro 2: Imágenes empleadas en cada práctica.

Práctica	Imágenes
1	`im1.jpg` a `im6.jpg` (escala de grises) y `im4.jpg` para la medida de las botellas
2	`im7.jpg` a `im9.jpg` (color), `im1.jpg`, `colorWheel.png` e `im10.jpg`
3	Dos imágenes del conjunto `im13.jpg` a `im20.jpg`, del mismo tamaño
4	`im6.jpg` (blíster de pastillas)
5	`im12.jpg` (objetos de colores)

3.3. Organización del código

Cada práctica se entrega como un script independiente, con el nombre que indica el enunciado (`practica_0.m` a `practica_5.m`). Los scripts se ejecutan situándose en la carpeta que contiene las imágenes, ya que estas se cargan por nombre relativo:

```
cd ~/Documentos/Universidad/HARDWARE/BLOQUE1
practica_1
```

4. Práctica 0: Introducción a MATLAB

4.1. Planteamiento

La primera práctica no trabaja todavía con imágenes. Su objetivo es adquirir soltura con las estructuras de datos de MATLAB que se van a necesitar después: vectores, matrices, concatenación en las distintas dimensiones, indexación, máscaras lógicas, *cell arrays* y medida de tiempos de ejecución.

4.2. Concatenación en las tres dimensiones

A partir de los vectores fila

$$v_1 = (1,\ 2,\ 3), \qquad v_2 = (3,\ 2,\ 1), \qquad v_3 = (5,\ 7,\ -1)$$

se pide construir tres variables concatenándolos en cada una de las tres dimensiones del espacio. La función que permite hacerlo de forma explícita es `cat`, cuyo primer argumento es la dimensión:

```
M1 = cat(1, v1, v2, v3);   % dimension 1: apilado vertical
M2 = cat(2, v1, v2, v3);   % dimension 2: concatenacion horizontal
M3 = cat(3, v1, v2, v3);   % dimension 3: matriz 3D
```

El resultado es una matriz de 3×3 en el primer caso, un vector fila de 1×9 en el segundo y una matriz tridimensional de $1 \times 3 \times 3$ en el tercero, donde cada capa es uno de los vectores originales.

Cuadro 3: Tamaño de las variables obtenidas.

Variable	Tamaño	Interpretación
`M1`	3×3	Los tres vectores como filas
`M2`	1×9	Los tres vectores uno a continuación de otro
`M3`	$1 \times 3 \times 3$	Los tres vectores como capas

4.3. Operaciones con matrices

Sobre las matrices

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \qquad B = \begin{pmatrix} 9 & 8 & 7 \\ 6 & 5 & 4 \\ 3 & 2 & 1 \end{pmatrix}$$

se pide el producto componente a componente, que en MATLAB se escribe con el operador punto (`A .* B`), y no con `A * B`, que sería el producto matricial habitual. La diferencia entre ambos operadores es esencial en el tratamiento de imágenes, donde casi todas las operaciones se aplican píxel a píxel.

También se extraen submatrices mediante indexación por filas y columnas (`A(1:2, 2:3)`) y se construye una máscara lógica que vale 1 en las posiciones donde aparecen los valores 2 o 3. Multiplicar la máscara por la matriz original es la forma habitual de aislar esos elementos: las posiciones que no cumplen la condición se multiplican por cero.

4.4. Búsqueda de valores y matrices aleatorias

Para localizar la posición del elemento menor de una matriz no basta con `min`, que devuelve el valor mínimo de cada columna. Es necesario combinarlo con `find`, que devuelve los índices donde se cumple una condición:

```
vector_menor = min(E);          % minimo de cada columna
menor = min(vector_menor);      % minimo global
[filas, cols] = find(E == menor);
```

El último apartado genera una matriz de 1000×1000 con valores aleatorios uniformes en el intervalo $[0, 10)$ mediante `rand`, se queda con los valores comprendidos entre 4 y 5 y representa la fila 500 con `stem`. El tiempo de ejecución se mide con `tic` y `toc`, convirtiendo el resultado a milisegundos.

Respuesta. El uso de `rand(1000,1000)*10` genera valores en $[0, 10)$, mientras que la expresión `rand*10 + 1` los generaría en $[1, 11)$, fuera del enunciado. El tiempo medido con `tic`/`toc` incluye la creación de la matriz, el cálculo de la máscara lógica, el producto y la representación gráfica, y se expresa en milisegundos multiplicando por 1000.

5. Práctica 1: Procesado básico de imágenes en escala de grises

5.1. Planteamiento

El script debe pedir al usuario el nombre de una imagen en escala de grises, comprobar que el fichero existe y que realmente no tiene tres canales, y a continuación aplicar sobre ella las operaciones básicas de procesado: histograma, brillo, ecualización, umbralización, suavizado, gradiente y realce. Todas las figuras deben estar etiquetadas.

5.2. Validación de la entrada

La comprobación se hace en un bucle `while` que se repite mientras la condición de error sea cierta. La función `exist(nombre,'file')` indica si el fichero está en el directorio de trabajo, y el número de canales se obtiene con el tercer elemento devuelto por `size`:

```
nombre = input('Nombre de la imagen: ', 's');
while ~exist(nombre, 'file') || size(imread(nombre), 3) ~= 1
    disp('El fichero no existe o la imagen es en color');
    nombre = input('Nombre de la imagen: ', 's');
end
A = im2double(imread(nombre));
```

La conversión a `double` con `im2double` es importante: las funciones de MATLAB devuelven imágenes en `uint8` (rango 0–255) al leerlas del disco, y trabajar en ese formato con operaciones aritméticas provoca desbordamientos y saturaciones. Con `double` el rango es $[0, 1]$ y las operaciones son las mismas que se plantean en teoría.

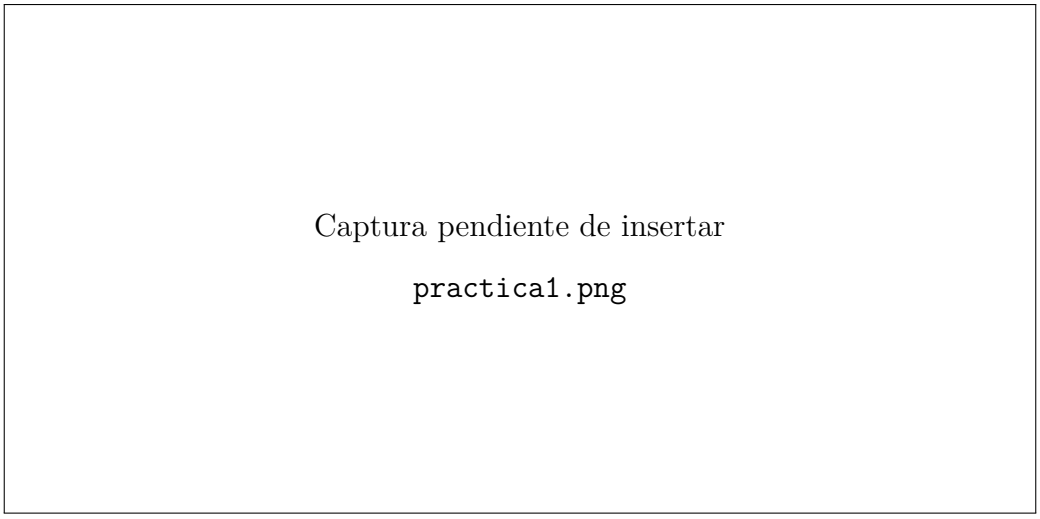


Figura 1: Imagen de partida y su histograma.

5.3. Brillo

Sumar una constante a todos los píxeles desplaza el histograma sin modificar su forma. El problema es que los valores pueden salir del rango válido, por lo que hay que recortarlos:

```
brillo = input('Brillo (-255 a 255): ');
B = max(min(A + brillo/255, 1), 0);
```

La división por 255 convierte el valor introducido por el usuario, que se pide en la escala 0–255 propia de `uint8`, al rango $[0, 1]$. El `min` y el `max` recortan por arriba y por abajo, respectivamente, evitando que la imagen se sature a blanco o a negro cuando se piden valores grandes.

5.4. Ecualización del histograma

La ecualización redistribuye los niveles de gris para que el histograma resultante sea lo más plano posible, aumentando el contraste en las zonas donde la imagen original concentraba muchos píxeles en pocos niveles. MATLAB la implementa en una sola función, `histeq`, y el resultado se compara con la imagen de partida observando los dos histogramas.

5.5. Umbralización

Se pide eliminar los píxeles cuya intensidad sea menor que un valor escogido por el usuario. La forma compacta de hacerlo es con indexación lógica:

```
D = A;
```

```
D(A < umbral/255) = 0;
```

La condición `A <umbral/255` es una matriz de valores lógicos del mismo tamaño que la imagen; al utilizarla como índice se seleccionan exactamente esos píxeles, que se igualan a cero.

5.6. Suavizado

Se aplican dos filtros espaciales de vecindad, el de media y el gaussiano. El primero promedia todos los píxeles de la ventana y el segundo los pondera con una campana de Gauss, lo que produce un desenfoque más natural y preserva mejor los bordes:

```
% tam y sigma se piden por teclado con input
Media = imfilter(A, fspecial('average', [tam tam]));
Gauss = imfilter(A, fspecial('gaussian', [tam tam], sigma));
```

El tamaño de la ventana debe ser impar para que exista un píxel central; con un valor par, `fspecial` desplaza el centro y el filtrado deja de ser simétrico.

Captura pendiente de insertar

`practica1-suavizado.png`

Figura 2: Suavizado de media y gaussiano.

5.7. Gradiente

El gradiente mide la variación local de intensidad y es la base de la detección de bordes. Se calcula con los operadores de Prewitt, que son dos máscaras de 3×3, una para la dirección horizontal y otra para la vertical:

```
Hx = fspecial('prewitt');
Hy = Hx';
Gx = imfilter(A, Hx);
Gy = imfilter(A, Hy);
G = 0.5 * (abs(Gx) + abs(Gy));
```

El módulo del gradiente se aproxima con la media de los valores absolutos de las dos componentes. El factor 0,5 es necesario porque, al sumar dos derivadas direccionales, el resultado puede duplicar el rango dinámico de la imagen.

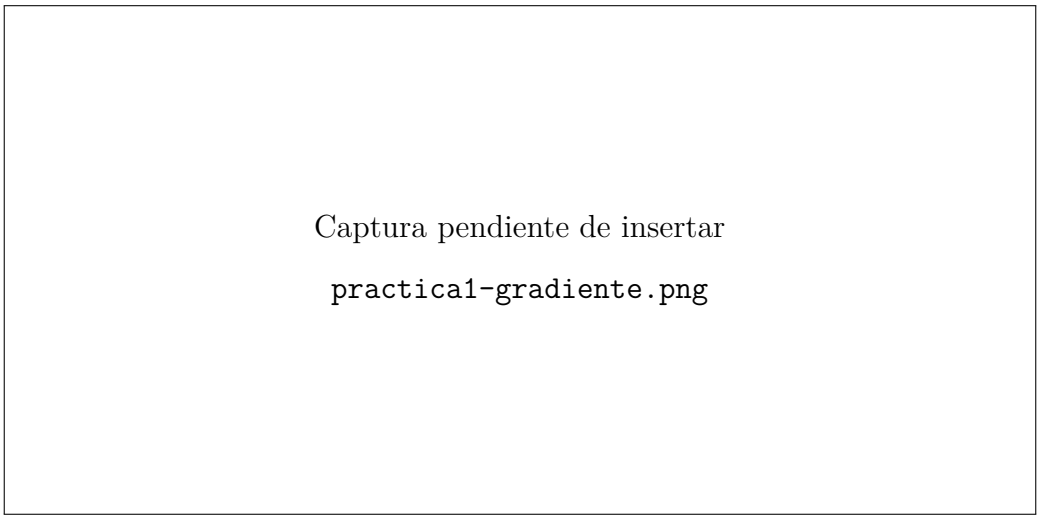


Figura 3: Módulo del gradiente y sus dos componentes.

5.8. Realce

El realce consiste en sumar a la imagen original el gradiente, que refuerza los bordes, o restarle el laplaciano, que es el operador de segunda derivada. El laplaciano se obtiene con `fspecial('laplacian')`; al restarlo se acentúan los cambios bruscos de intensidad:

```
L = imfilter(A, fspecial('laplacian'));
R1 = A + G;
R2 = A - L;
```

5.9. Medida de las botellas de `im4.jpg`

El último apartado es el más interesante desde el punto de vista de la inspección automática. La imagen `im4.jpg` es una captura de una cinta transportadora con botellas, y en ella se han recogido 90 cm de cinta. Se pide utilizar el gradiente para estimar la separación entre botellas y su anchura.

El procedimiento es el siguiente. Se calcula el gradiente de la imagen completa y se extrae la fila central, que es la que corta todas las botellas por el mismo punto. Sobre esa fila, los bordes de cada botella aparecen como picos de gradiente:

```
I = im2double(imread('im4.jpg'));
Gx = imfilter(I, Hx);
Gy = imfilter(I, Hy);
G = 0.5 * (abs(Gx) + abs(Gy));

```

```
[filas, columnas] = size(I);
fila = G(ceil(filas/2), :);
plot(fila)

pos  = find(fila > 0.4);       % posiciones de los picos
dist = diff(pos);              % distancia entre picos consecutivos
dist = dist(dist > 3);         % se descartan los picos pegados
```

El filtro `dist >3` es imprescindible: sobre un mismo borde el gradiente supera el umbral en varios píxeles consecutivos, lo que produce distancias de 1 o 2 píxeles que no corresponden a ningún objeto. Sin ese filtro, el mínimo sería 1 píxel y la separación calculada sería prácticamente cero.

Conocida la correspondencia entre píxeles y centímetros (498 px = 90 cm, es decir, 5,53 px/cm), las distancias se convierten a unidades reales.

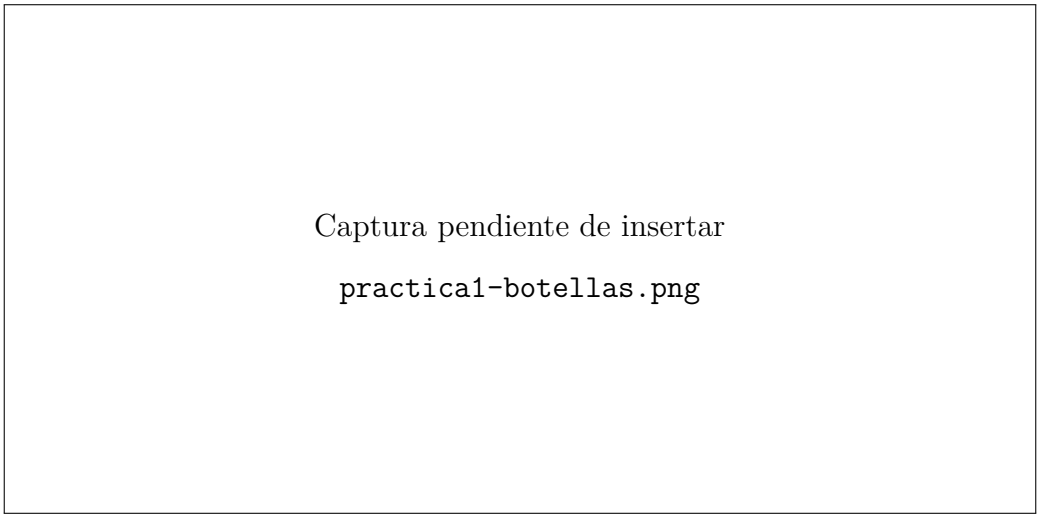


Figura 4: Gradiente de la fila central de `im4.jpg`.

Respuesta. En la fila central se detectan 21 picos que, tras descartar los que están pegados, dejan nueve distancias significativas: 24, 25, 25, 31, 52, 56, 90, 91 y 91 píxeles.
Las cuatro distancias pequeñas (24–31 px) son los huecos entre botellas consecutivas. Las tres grandes (90–91 px) son botellas completas. Las dos intermedias (52 y 56 px) corresponden a las dos botellas de los extremos, que aparecen cortadas por el borde de la imagen.
Por tanto:

- **Separación aproximada entre botellas:** 24 px $\times$ 90/498 = **4,34 cm**.
- **Ancho aproximado de cada botella:** 91 px $\times$ 90/498 = **16,45 cm**.

La coherencia del resultado se comprueba sumando todas las distancias: las nueve significativas suman 485 px y las once descartadas por el filtro suman 12 px, es decir, 497 px de los 498 de la imagen. La imagen queda explicada en su totalidad por la secuencia hueco–botella–hueco–botella.

6. Práctica 2: Procesado de imágenes en color

6.1. Planteamiento

Una imagen en color es una matriz de tres dimensiones en la que la tercera dimensión son los canales. El script pide una imagen en color, comprueba que efectivamente tiene tres canales, y trabaja con las distintas representaciones del color para seleccionar objetos y recolorear imágenes en escala de grises.

6.2. Canales RGB

La representación RGB descompone el color en sus tres componentes primarias. Separarlas es tan sencillo como indexar la tercera dimensión:

```
R = A(:,:,1);
G = A(:,:,2);
B = A(:,:,3);
```

Cada canal se muestra junto a su histograma. En una imagen con dominante de color, el histograma del canal correspondiente aparece desplazado hacia valores altos.

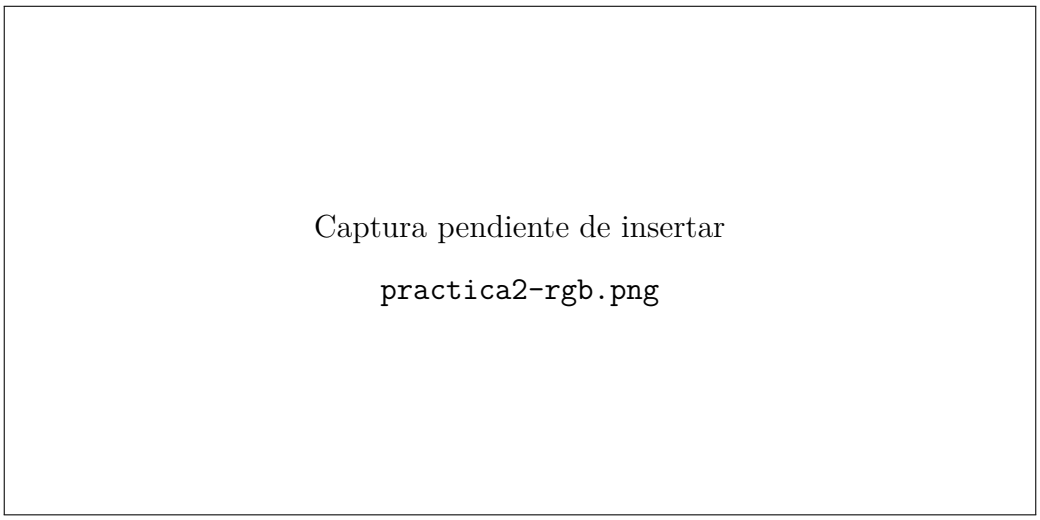


Figura 5: Canales R, G y B con sus histogramas.

6.3. Espacios HSV e YCbCr

El espacio HSV separa la información de color (tono y saturación) de la luminosidad (valor). Esto lo hace mucho más adecuado que RGB para segmentar por color, porque el tono de un objeto se mantiene relativamente estable aunque cambien las condiciones de iluminación.

El espacio YCbCr, utilizado en vídeo y en compresión JPEG, separa igualmente la luminancia (Y) de las dos componentes de crominancia (Cb y Cr). Es el que se emplea en la práctica 3 para construir la mascarilla del *chroma key*.

```
HSV = rgb2hsv(A);
H = HSV(:,:,1); S = HSV(:,:,2); V = HSV(:,:,3);

YCC = rgb2ycbcr(A);
Y = YCC(:,:,1); Cb = YCC(:,:,2); Cr = YCC(:,:,3);
```

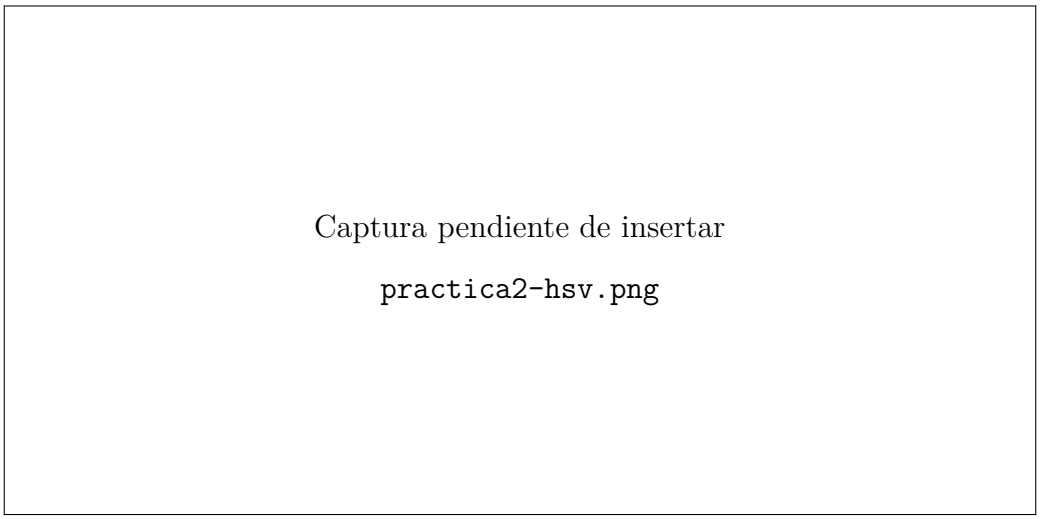


Figura 6: Canales H, S y V con sus histogramas.

6.4. Selección de un color con `impixel`

La función `impixel` permite seleccionar píxeles con el ratón y devuelve su valor de color. A partir del tono elegido se construye una máscara y se anula la saturación del resto de la imagen, de forma que solo el color seleccionado permanece en color y el resto se ve en gris:

```
imshow(A)
[x, y, P] = impixel;
h = rgb2hsv(P);
h = h(1);

d = abs(H - h);
M = min(d, 1 - d) < 0.02;    % el tono es circular
Res = hsv2rgb(cat(3, H, S .* M, V));
```

El detalle importante es el cálculo de la distancia de tono. El tono es una magnitud circular que va de 0 a 1 y vuelve a empezar: el rojo está simultáneamente en 0 y en 1. Una resta directa `abs(H - h)` fallaría para los tonos próximos al rojo, porque consideraría que 0,99 y 0,01 están muy alejados cuando en realidad son casi el mismo

color. La expresión `min(d, 1-d)` mide la distancia por el arco más corto y resuelve el problema.

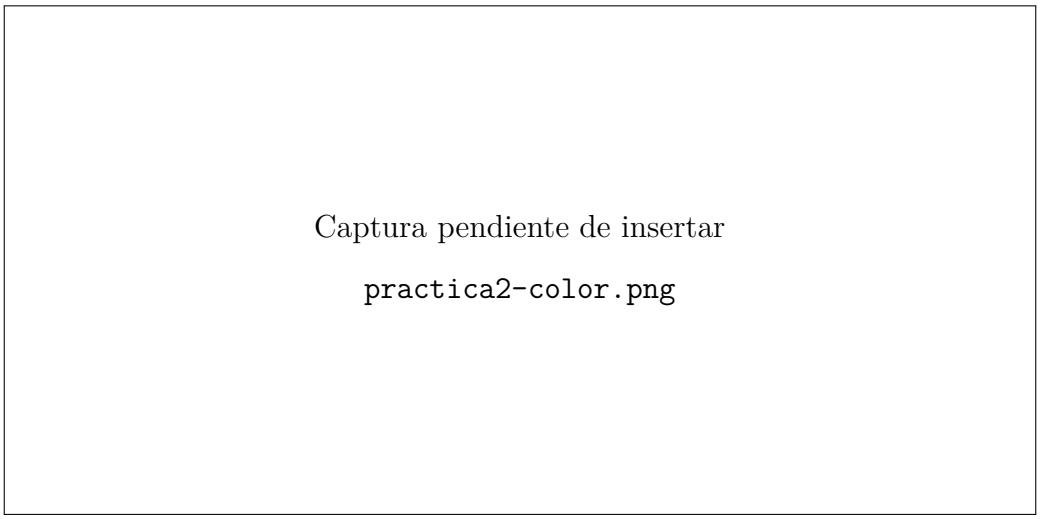


Figura 7: Selección de un color con `impixel`.

6.5. Mapa de color

Una imagen en escala de grises se puede colorear aplicando un mapa de color, es decir, una tabla que asigna un color RGB a cada nivel de intensidad. El usuario elige el mapa por teclado y se aplica con `ind2rgb`:

```
Gris = imread('im1.jpg');
mapa = input('Mapa de color (jet, hot, bone, ...): ', 's');
m = feval(mapa, 256);
Color = ind2rgb(Gris, m);
```

Un detalle que conviene tener en cuenta es que `colormap` actúa sobre la figura completa y no solo sobre el eje actual, por lo que al aplicarlo hay que restringirlo al eje de la derecha con `colormap(gca, m)`. De lo contrario, la imagen que debería verse en grises aparece también coloreada.

Captura pendiente de insertar

`practica2-mapa.png`

Figura 8: Imagen en grises y con mapa de color.

6.6. Color a partir de la rueda HSV

La segunda forma de colorear no utiliza un mapa predefinido, sino que toma el tono y la saturación de un punto elegido sobre la imagen `colorWheel.png`, y utiliza la imagen en grises como luminosidad:

```
Rueda = im2double(imread('colorWheel.png'));
imshow(Rueda)
[x, y, P] = impixel;
hsv_p = rgb2hsv(P);

Gd = im2double(Gris);
Hh = ones(size(Gd)) * hsv_p(1);
Ss = ones(size(Gd)) * hsv_p(2);
Color2 = hsv2rgb(cat(3, Hh, Ss, Gd));
```

El resultado es una imagen monocroma teñida con el color elegido, conservando toda la información de textura de la imagen original en el canal V.

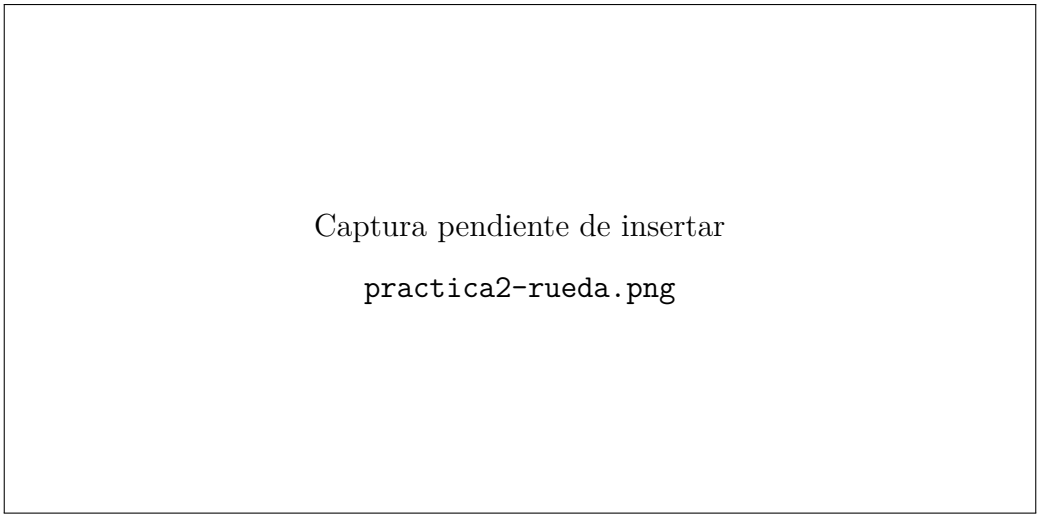


Figura 9: Resultado del coloreado con HSV.

6.7. Diámetro de la moneda de `im10.jpg`

El último apartado pide estimar el diámetro de la moneda que aparece en la imagen, sabiendo que la superficie representada es de unos 4800 mm^2. El procedimiento es segmentar la moneda y aplicar una regla de tres entre el área en píxeles y el área real:

```
M10 = im2double(imread('im10.jpg'));
Gm = rgb2gray(M10);

L = (Gm < 0.9);                          % la moneda es oscura, el
    fondo blanco
pixeles_moneda = sum(sum(L));
[filas, columnas, z] = size(M10);

area = pixeles_moneda / (filas*columnas) * 4800 / 2;
diametro = 2 * sqrt(area / pi);
```

Como la imagen se ha convertido previamente a `double`, tanto `rgb2gray` como la comparación trabajan en el rango $[0, 1]$, de modo que el umbral 0,9 es correcto. El umbral se elige alto de forma intencionada: el fondo de la imagen es blanco puro (valor 1) y las monedas, con su sombra, quedan por debajo.

La división entre dos responde a que en la imagen aparecen las dos caras de la misma moneda, de modo que el área calculada corresponde al doble del área de una cara.

Captura pendiente de insertar

`practica2-moneda.png`

Figura 10: Segmentación de la moneda de `im10.jpg`.

Respuesta. La segmentación por umbral deja un 48,9 % de los píxeles por debajo de 0,9, que corresponden a las dos monedas y a su sombra. Aplicando la regla de tres sobre los 4800 mm^2 de superficie total y dividiendo entre dos caras, el área de una cara resulta

$$A = \frac{0{,}489 \cdot 4800}{2} \approx 1173 \text{ mm}^2$$

y el diámetro equivalente es

$$d = 2\sqrt{\frac{A}{\pi}} \approx \mathbf{38{,}65 \text{ mm}}.$$

El valor es notablemente mayor que el de una moneda real de euro, cuyo diámetro está entre 16 y 26 mm. La diferencia se explica porque la escala de 4800 mm^2 es aproximada y porque el área segmentada incluye la sombra proyectada, que agranda el objeto. El resultado debe interpretarse, por tanto, como una estimación del orden de magnitud y no como una medida exacta.

7. Práctica 3: Chroma Key

7.1. Planteamiento

El *chroma key* es la técnica que permite sustituir el fondo de una imagen por otro arbitrario. Se graba al sujeto sobre un fondo de color uniforme (normalmente verde o azul) y se genera una mascarilla que vale 1 en el sujeto y 0 en el fondo. La composición final es una mezcla ponderada de las dos imágenes:

$$I_{\text{final}} = \alpha \cdot I_{\text{objeto}} + (1 - \alpha) \cdot I_{\text{fondo}}$$

El script pide los nombres de las dos imágenes y calcula la mascarilla por dos métodos

distintos.

7.2. Mascarilla por distancia de color

El primer método compara el color de cada píxel con el color medio del fondo. Se calcula la distancia euclídea en el espacio RGB y se umbraliza: los píxeles próximos al color del fondo se consideran fondo.

7.3. Mascarilla por diferencia de color

El segundo método utiliza la diferencia entre canales, que es más robusta frente a variaciones de iluminación porque la componente de brillo afecta por igual a los tres canales y se cancela parcialmente al restarlos.

7.4. Ajuste de la mascarilla

La mascarilla obtenida directamente suele tener bordes duros y píxeles sueltos. El ajuste consiste en suavizar el contorno para que la transición no sea abrupta, lo que evita el efecto de silueta recortada.

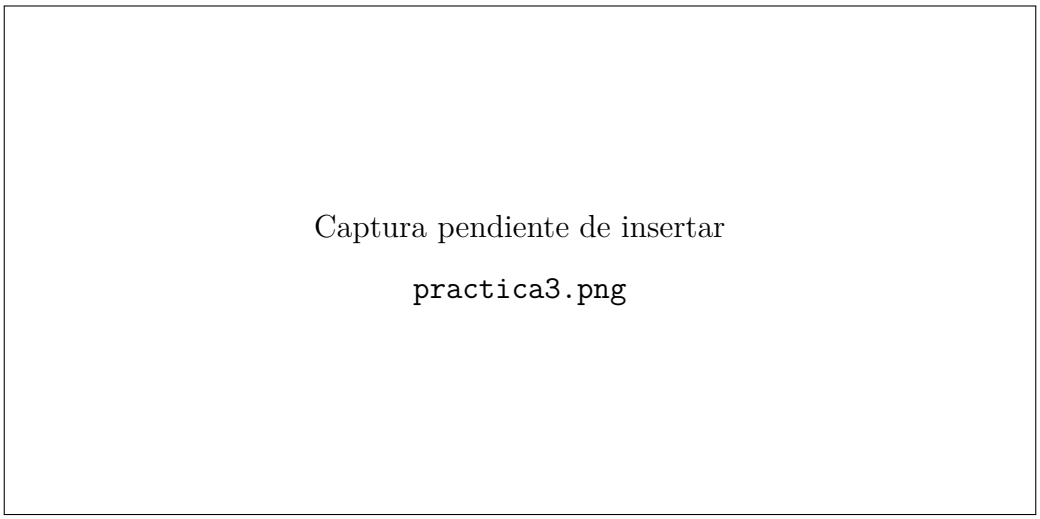


Figura 11: Resultado del *chroma key* por los dos métodos.

Respuesta. Se han utilizado las imágenes `im13.jpg` como objeto y `im14.jpg` como fondo nuevo, ambas de las mismas dimensiones.
La técnica de distancia de color separa correctamente el objeto del fondo verde, pero deja algunos píxeles del contorno mal clasificados, que se corrigieron ajustando la mascarilla. La técnica de diferencia de color resulta algo más estable frente a las sombras del fondo.
(Completar con la comparación de los resultados obtenidos en las dos figuras y con los valores de umbral utilizados.)

8. Práctica 4: Imágenes binarias

8.1. Planteamiento

La imagen binaria es el resultado de reducir la información de la imagen a dos valores: objeto y fondo. Sobre ella se pueden aplicar operaciones morfológicas y de etiquetado que permiten contar objetos y medir sus propiedades geométricas. La práctica trabaja con `im6.jpg`, un blíster de pastillas.

8.2. Segmentación

El histograma de la imagen muestra dos poblaciones bien separadas, la de las pastillas y la del blíster, lo que permite separarlas con un umbral. Para que la segmentación sea robusta se combinan varias etapas, y es conveniente visualizar el resultado de cada una:

1. Umbralización para obtener la imagen binaria.
2. Relleno de huecos con `imfill`, para que cada pastilla sea una región compacta y no un anillo.
3. Apertura morfológica para eliminar los objetos de pequeño tamaño que no son pastillas.
4. Etiquetado de las regiones conexas con `bwlabel`.

8.3. Visualización de la matriz de etiquetas

La matriz devuelta por `bwlabel` contiene un entero distinto en cada objeto, y 0 en el fondo. Para representarla se normaliza con `mat2gray` y se aplica un mapa de color con un número de colores igual al número de objetos más uno:

```
[etiquetas, n] = bwlabel(binaria);
E = mat2gray(etiquetas);
imshow(E, jet(n+1)), colorbar
```

Es importante ajustar el número de colores: con el mapa por defecto, dos objetos con etiquetas próximas pueden recibir colores casi idénticos y confundirse.

8.4. Rectángulos de área mínima y enumeración

La función `regionprops` devuelve, para cada región, sus propiedades geométricas. Las que se utilizan aquí son `BoundingBox`, para dibujar el rectángulo de área mínima, y `Centroid`, para colocar el número del objeto con `text`:

```
props = regionprops(etiquetas, 'BoundingBox', 'Centroid');
for k = 1:numel(props)
    rectangle('Position', props(k).BoundingBox, 'EdgeColor', 'r');
    c = props(k).Centroid;
    text(c(1), c(2), num2str(k), 'Color', 'r');
end
```

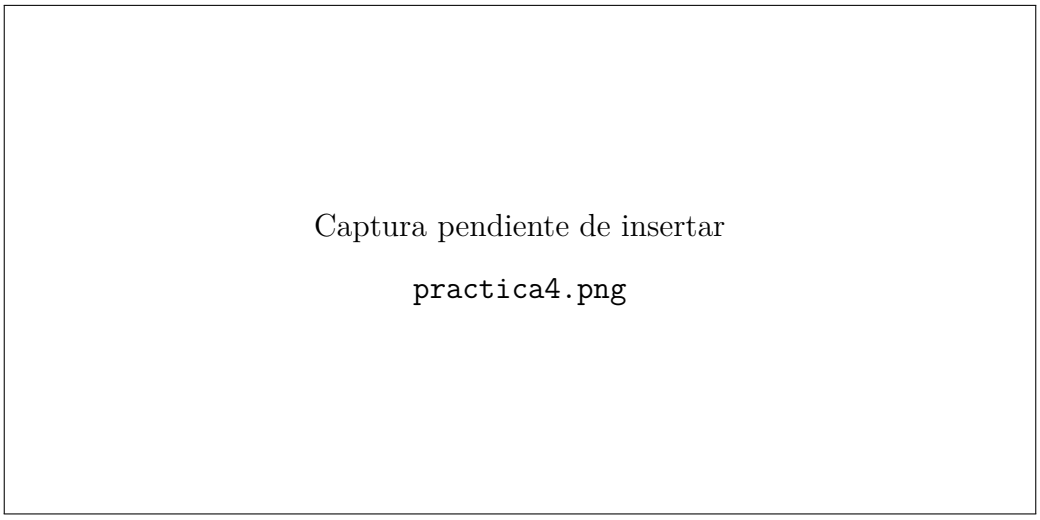


Figura 12: Pastillas detectadas con su rectángulo y su número.

8.5. Ranuras horizontales

El último apartado pide contar las ranuras horizontales del blíster y mostrarlas aisladas. Se consiguen seleccionando, de entre los objetos detectados, aquellos cuya relación de aspecto es muy alargada en horizontal, es decir, aquellos en los que la anchura del rectángulo es mucho mayor que su altura.

Respuesta. El número aproximado de ranuras horizontales es *(completar)*.
Para aislarlas se ha filtrado la lista de regiones por su relación de aspecto w/h, conservando únicamente las que superan un valor umbral. La imagen resultante contiene solo las ranuras, sin las pastillas.

---

9. Práctica 5: Reconocimiento y clasificación

9.1. Planteamiento

La última práctica del bloque da un paso más: no basta con detectar objetos, hay que describirlos numéricamente para poder clasificarlos. Se trabaja con `im12.jpg`, que

contiene objetos de distintos colores y formas, algunos de los cuales son falsos positivos que deben descartarse.

9.2. Segmentación y etiquetado

Al tratarse de una imagen en color, la segmentación se hace sobre uno de los canales o sobre una combinación de ellos, buscando el que ofrezca mayor contraste entre los objetos y el fondo.

9.3. Factor de forma

El factor de forma es un descriptor geométrico que mide cuánto se parece un objeto a un círculo. Se calcula a partir del área y del perímetro:

$$\text{factor de forma} = \frac{4\pi A}{P^2}$$

Vale 1 para un círculo perfecto y disminuye a medida que la forma se aleja de él. Se representa con `stem`, un objeto por muestra.

9.4. Textura

La textura se caracteriza mediante la entropía, que mide la cantidad de información o de desorden de la región. Una región uniforme tiene entropía baja y una región con mucho detalle tiene entropía alta. Al igual que el factor de forma, se representa con `stem`.

9.5. Espacio de características

Representando cada objeto como un punto en un plano cuyos ejes son el factor de forma y la textura, los objetos de la misma clase tienden a agruparse. Los falsos positivos, es decir, los elementos circulares que no son reales, aparecen como puntos alejados del grupo principal.

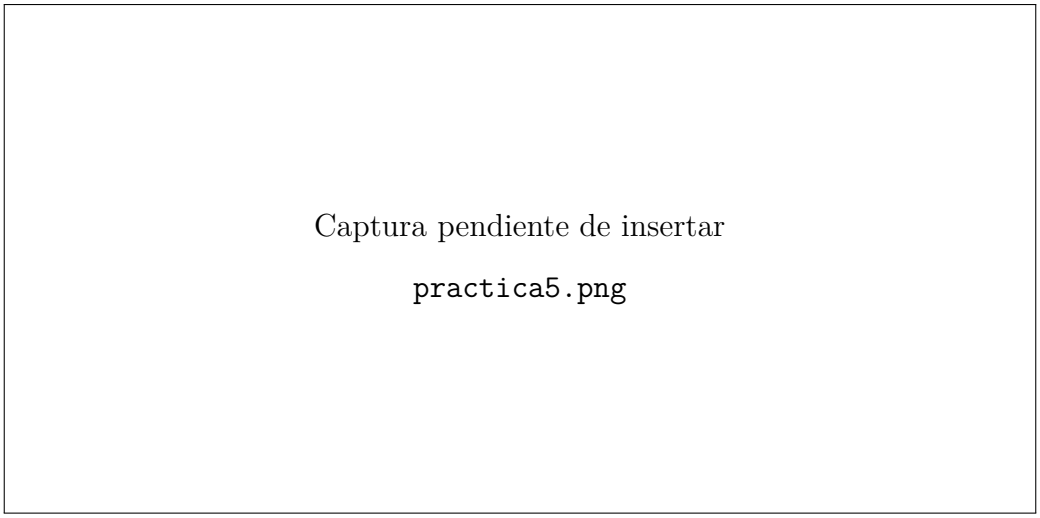


Figura 13: Clasificación de los objetos en el espacio de características.

Respuesta. Los objetos reales se concentran en una zona del plano con factor de forma próximo a 1 y valores de entropía similares entre sí. Los elementos circulares que no son reales quedan fuera de esa concentración y se han señalado sobre la imagen original.
(Completar con el número de objetos detectados y con el criterio concreto utilizado para separar los objetos reales de los falsos positivos.)

10. Reto 1: ¿Cuántos euros hay? (opcional)

El reto propone aplicar de forma conjunta lo aprendido en las prácticas 4 y 5 para contar el dinero que aparece en una imagen. El procedimiento sería segmentar las monedas, medir el diámetro de cada una y clasificarla en función de su tamaño, ya que cada valor de euro tiene un diámetro característico.

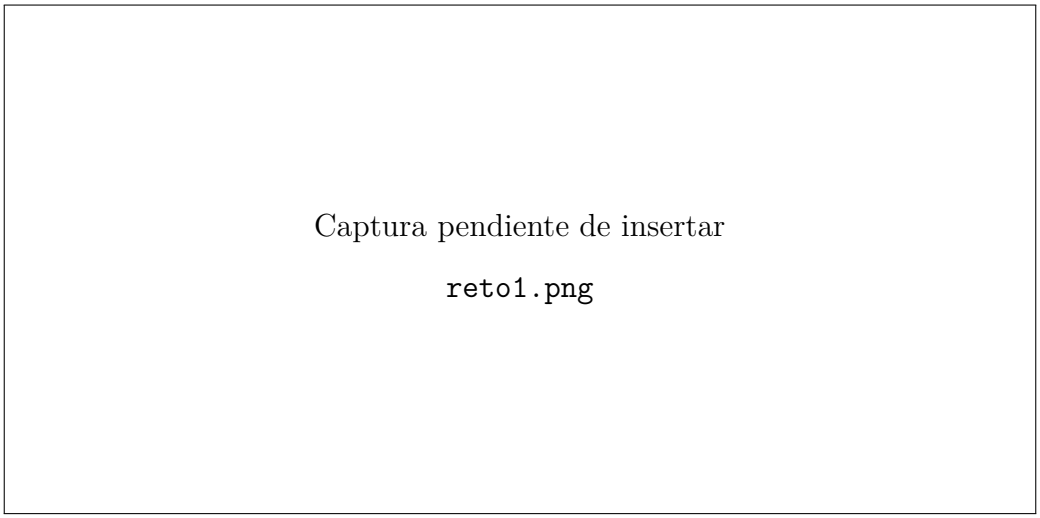


Figura 14: Monedas detectadas y clasificadas por su diámetro.

Respuesta. *(Apartado opcional. Completar con el resultado obtenido o indicar que no se ha realizado.)*

11. Conclusiones

Las prácticas del bloque permiten recorrer el procesamiento digital de imagen desde las operaciones más elementales hasta la clasificación de objetos. El recorrido deja varias ideas claras.

En primer lugar, la representación de la imagen como matriz condiciona por completo el programa: casi todas las operaciones se reducen a aritmética elemento a elemento, y el control del rango de valores (`uint8` frente a `double`) es la principal fuente de errores.

En segundo lugar, la elección del espacio de color no es indiferente. Trabajar en HSV o en YCbCr simplifica enormemente la segmentación por color frente a hacerlo en RGB, y el tratamiento del tono como magnitud circular es un detalle que, si se pasa por alto, arruina la selección de los colores próximos al rojo.

En tercer lugar, la medida de magnitudes reales exige establecer una escala y ser consciente de sus limitaciones: el resultado de las botellas es coherente porque la escala de 90 cm está bien definida, mientras que el de la moneda solo es una estimación, porque la superficie de 4800 mm^2 es aproximada y la sombra agranda el objeto segmentado.

Por último, las prácticas 4 y 5 muestran que la detección de objetos no termina en la umbralización: hacen falta operaciones morfológicas para limpiar la imagen binaria, etiquetado para separar objetos y descriptores numéricos para poder clasificarlos y descartar los falsos positivos.

12. Contribución de los integrantes

Cuadro 4: Contribución de los integrantes del grupo.

Estudiantes	**Contribución**
Nombre y apellidos 1	*(Describir las tareas realizadas.)*
Nombre y apellidos 2	*(Describir las tareas realizadas.)*

13. Declaración sobre el uso de herramientas de inteligencia artificial

Se utilizó una herramienta de inteligencia artificial generativa como apoyo para la revisión de la redacción y la resolución de dudas puntuales de programación. Los autores del trabajo revisaron, comprendieron y validaron el código y los resultados presentados.

14. Referencias

Referencias

[1] C. Represa Pérez, *Procesamiento digital de imagen y vídeo. Hardware de Aplicación Específica*, v5.0, Área de Tecnología Electrónica, Universidad de Burgos, 2026.

[2] R. C. Gonzalez y R. E. Woods, *Digital Image Processing*, 4.ª ed., Pearson, 2018.

[3] The MathWorks, Inc., *Image Processing Toolbox User's Guide*. https://www.mathworks.com/help/images/

A. Código fuente y repositorio

Los scripts desarrollados se encuentran en la carpeta `BLOQUE1` del repositorio del grupo:

Cuadro 5: Relación de scripts entregados.

Fichero	Contenido
`practica_0.m`	Introducción a MATLAB
`practica_1.m`	Procesado básico de imágenes en grises
`practica_2.m`	Procesado de imágenes en color
`practica_3.m`	*Chroma key*
`practica_4.m`	Imágenes binarias
`practica_5.m`	Reconocimiento y clasificación
`reto.m`	Reto 1 (opcional)

Para ejecutar cualquier script es necesario situarse en la carpeta que contiene las imágenes, ya que estas se cargan por nombre relativo:

```
cd BLOQUE1
practica_1
```

B. Material adicional

Las capturas de pantalla de las figuras generadas por cada script se incluyen junto con este documento. Los nombres de los ficheros coinciden con los referenciados en las figuras del texto.